

学号： 2106410119



沈阳建筑大学

大数据技术基础 作业

2023 年秋季学期

学 院： 计算机科学与工程学院

专业班级： 计算机 2101 班

姓 名： 陈芷涵

第一次

page 33

2.1 “大润发”、“沃尔玛”、“好德”和“农工商”四个超市都卖苹果、梨、香蕉、桔子和芒果五种水果。使用 NumPy 的 ndarray 实现以下功能。

- (1) 创建 2 个一维数组分别存储超市名称和水果名称；
- (2) 创建 1 个 4×5 的二维数组存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成；
- (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元；
- (4) “农工商”水果大减价，所有水果价格减少 2 元；
- (5) 统计四个超市苹果和芒果的销售均价；
- (6) 找出桔子价格最贵的超市名称（不是编号）。

程序代码

```
# 提示用户
name1=input("请输入姓名: ")
# 创建一个空的字符串列表
name_list = []
# 输入若干姓名，直到用户输入空行为止
while True:
    name1 = input("请输入姓名: ")
    if name1 == "":
        break
    name_list.append(name1)

m = input("请输入查找姓名: ")

# 检查姓名是否存在于列表中
for i in name_list:                                #遍历列表。分两种情况：m 在 names 之
    if i == m:                                       内与 names 之外
        print('列表中存在！')
    else:
        z=0
        z = z-1                                    #根据 z 值判断 m 是否在 names 列表之内。
        if z==0:                                    #若 m 不在 names 之内，z 值为 0
            print('列表中不存在！')
```

程序说明

1. 用户输入：用户首先被提示输入自己的姓名，并随后可以输入多个姓名，直到输入一个空行为止。
2. 姓名列表：所有输入的姓名将被存储在一个字符串列表 `name\_list` 中，用于后续的查找操作。
3. 查找姓名：用户被要求输入一个要查找的姓名（`m`）。
4. 查找操作：程序会检查输入的姓名是否存在于 `name\_list` 中。
5. 查找结果：根据查找结果，程序将输出相应的消息：

如果输入的姓名存在于列表中，程序将输出“列表中存在！”。

如果输入的姓名不存在于列表中，程序将输出“列表中不存在！”。

运行结果截图

```
请输入姓名：陈奕迅
请输入姓名：张杰
请输入姓名：周杰伦
请输入姓名：周迅
请输入姓名：
请输入查找姓名：张杰
列表中存在！
```

第二次

page 33

2.1 “大润发”、“沃尔玛”、“好德”和“农工商”四个超市都卖苹果、梨、香蕉、桔子和芒果五种水果。使用 NumPy 的 ndarray 实现以下功能。

- (1) 创建 2 个一维数组分别存储超市名称和水果名称；
- (2) 创建 1 个 4×5 的二维数组存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成；
- (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元；
- (4) “农工商”水果大减价，所有水果价格减少 2 元；
- (5) 统计四个超市苹果和芒果的销售均价；
- (6) 找出桔子价格最贵的超市名称（不是编号）。

程序代码

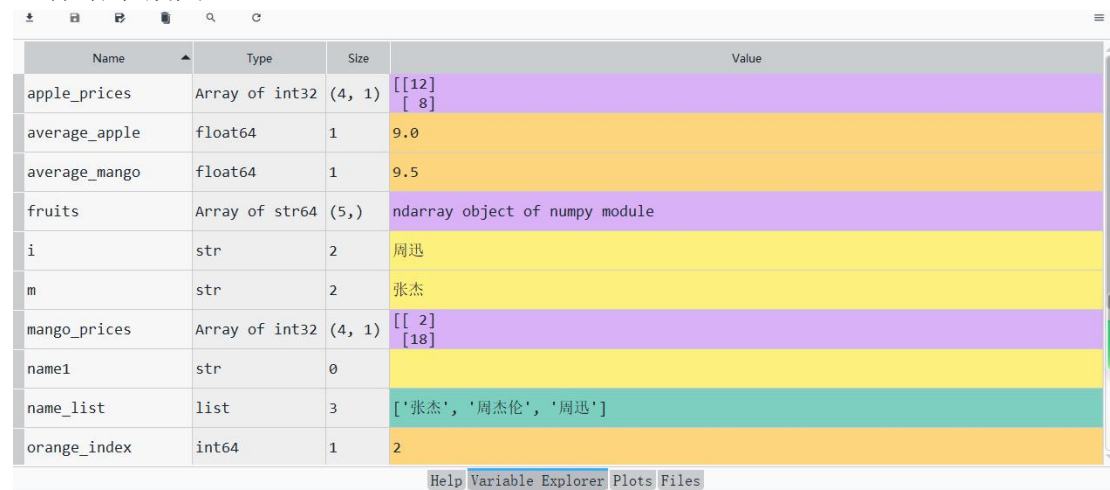
```
import numpy as np
# (1) 创建 1 维数组存储超市名称和水果名称
supermarkets = np.array(["大润发", "沃尔玛", "好德", "农工商"])
fruits = np.array(["苹果", "梨", "香蕉", "桔子", "芒果"])
# (2) 创建 4x5 的二维数组存储不同超市的水果价格
prices = np.random.randint(1, 20, size=(4, 5))
# (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元
prices[supermarkets == "大润发", fruits == "苹果"] += 1
prices[supermarkets == "好德", fruits == "香蕉"] += 1
# (4) “农工商”水果大减价，所有水果价格减少 2 元
prices[supermarkets == "农工商"] -= 2
# (5) 统计四个超市苹果和芒果的销售均价
apple_prices = prices[:, fruits == "苹果"]
average_apple = np.mean(apple_prices)
mango_prices = prices[:, fruits == "芒果"]
average_mango = np.mean(mango_prices)
print(f"苹果的销售均价: {average_apple}")
print(f"芒果的销售均价: {average_mango}")
# (6) 找出桔子价格最贵的超市名称
orange_index = np.argmax(prices[:, fruits == "桔子"])
orange_supermarket = supermarkets[orange_index]
```

```
print(f"桔子价格最贵的超市是: {orange_supermarket}")
```

程序说明

1. 创建数组: 首先, 程序创建两个一维数组, 分别用于存储超市名称和水果名称。然后, 通过随机数生成一个 4x5 的二维数组, 用于存储不同超市的水果价格。
2. 修改价格: 程序根据特定条件修改价格。
针对“大润发”的苹果和“好德”的香蕉, 将价格分别增加 1 元。
针对“农工商”超市的所有水果, 将价格减少 2 元。
3. 计算销售均价: 程序计算了四个超市中苹果和芒果的销售均价, 并输出结果。
4. 找出价格最贵的超市: 程序找出了桔子价格最高的超市, 并输出其名称。

运行结果截图



Name	Type	Size	Value
apple_prices	Array of int32	(4, 1)	[[12] [8]
average_apple	float64	1	9.0
average_mango	float64	1	9.5
fruits	Array of str64	(5,)	ndarray object of numpy module
i	str	2	周迅
m	str	2	张杰
mango_prices	Array of int32	(4, 1)	[[2] [18]
name1	str	0	
name_list	list	3	['张杰', '周杰伦', '周迅']
orange_index	int64	1	2

苹果的销售均价: 9.0

芒果的销售均价: 9.5

桔子价格最贵的超市是: 好德

第三次

三、page 63 3.2

根据银行储户的基本信息, 完成以下分析。

- 1) 从“bankpep.csv”文件中读取用户信息。
- 2) 查看储户的总数, 以及居住在不同区域的储户数。
- 3) 计算不同性别储户收入的均值和方差。
- 4) 统计接受新业务的储户中各类性别、区域的人数。
- 5) 将存款账户、接受新业务的值转化为数值型。
- 6) 分析收入、存款账户与接收新业务之间的关系。

```
import pandas as pd
```

```
import numpy as np
```

```
# 1. 从“bankpep.csv”文件中读取用户信息
```

```
df = pd.read_csv('C:/Users/wl198/.spyder-py3/bankpep.csv')
```

```
# 2. 查看储户的总数, 以及居住在不同区域的储户数
```

```
total_customers = len(df)
```

```
customers_by_region = df['region'].value_counts()
```

```
print(f"总储户数: {total_customers}")
```

```

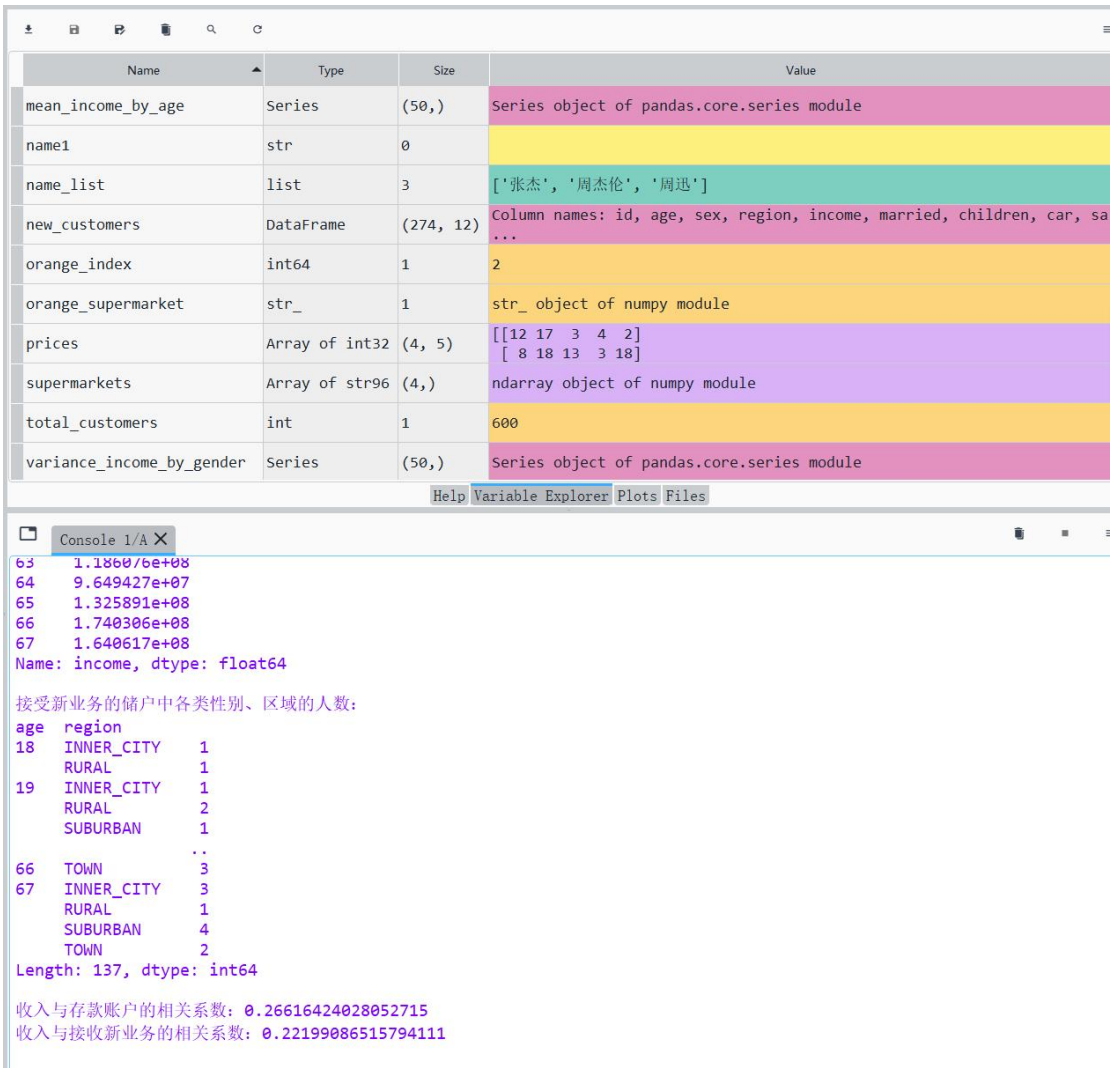
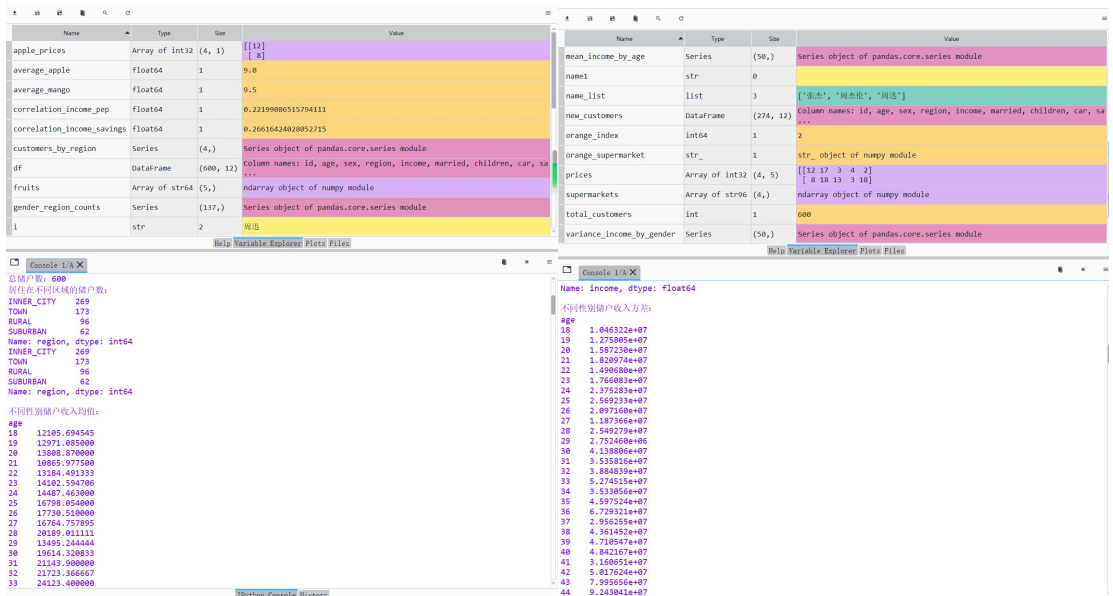
print("居住在不同区域的储户数: ")
print(customers_by_region)
# 3. 计算不同性别储户收入的均值和方差
mean_income_by_age = df.groupby('age')['income'].mean()
variance_income_by_gender = df.groupby('age')['income'].var()
print(customers_by_region)
print("\n 不同性别储户收入均值: ")
print(mean_income_by_age)
print("\n 不同性别储户收入方差: ")
# 4. 统计接受新业务的储户中各类性别、区域的人数
new_customers = df[df['pep'] == 'YES']
gender_region_counts = new_customers.groupby(['age', 'region']).size()
print(variance_income_by_gender)
print("\n 接受新业务的储户中各类性别、区域的人数: ")
# 5. 将存款账户、接受新业务的值转化为数值型
df['save_act'] = df['save_act'].map({'YES': 1, 'NO': 0})
df['pep'] = df['pep'].map({'YES': 1, 'NO': 0})
# 6. 分析收入、存款账户与接收新业务之间的关系
correlation_income_savings = df['income'].corr(df['save_act'])
correlation_income_pep = df['income'].corr(df['pep'])
print(gender_region_counts)
print(f"\n 收入与存款账户的相关系数: {correlation_income_savings}")
print(f"收入与接收新业务的相关系数: {correlation_income_pep}")

```

程序说明

1. 读取用户信息：程序首先从名为 "bankpep.csv" 的文件中读取银行储户的信息，并将其存储在一个 Pandas DataFrame 对象中。
2. 查看储户数量：程序计算总储户数量，并通过`value\_counts()`函数统计了居住在不同区域的储户数量。
3. 计算不同性别储户收入的均值和方差：程序以年龄为基准，计算了不同性别储户的收入均值和方差。
4. 统计接受新业务的储户数量：程序筛选出接受新业务的储户，并统计了这些储户中各类性别和区域的人数。
5. 将存款账户和接受新业务的值转化为数值型：程序将存款账户和接受新业务的标志从字符串（'YES'和'NO'）转换为数值型（1 和 0），以便进行相关性分析。
- 6.分析收入、存款账户和接收新业务之间的关系**：程序计算了收入与存款账户之间的相关系数和收入与接收新业务之间的相关系数，以帮助分析这些因素之间的关联性。

运行结果截图



第四次

二、page 86 可视化综合应用

文件 bankpep.csv 存放着银行储户的基本信息, 请通过绘图对这些客户数据进行探索性分析。

- 1) 客户年龄分布的直方图和密度图
- 2) 客户年龄和收入关系的散点图
- 3) 绘制散点图观察账户(年龄, 收入, 孩子数)之间的关系, 对角线显示直方图
- 4) 按区域展示平均收入的柱状图, 并显示标准差
- 5) 多子图绘制: 账户中性别占比饼图, 有车的性别占比饼图, 按孩子数的账户占比饼图
- 6) 各性别收入的箱须图

程序代码

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
# 读取数据
data = pd.read_csv('C:/Users/qingge/.spyde-py/bankpep.csv')
# 1) 客户年龄分布的直方图和密度图
data['age'].plot(kind='hist', bins=20, title='Customer age distribution')
data['age'].plot(kind='kde', title='Customer age density chart', style='k-')
plt.xlabel('Age')
plt.ylabel('Density')
plt.show()
# 2) 客户年龄和收入关系的散点图
data.plot(kind='scatter', x='age', y='income', title='Scatter plot of customer age and income relationship', marker='s', s=8)
plt.xlabel('Age')
plt.ylabel('Income')
plt.show()
# 3) 散点图观察账户(年龄, 收入, 孩子数)之间的关系, 对角线显示直方图
scatter_matrix = pd.plotting.scatter_matrix(data[['age', 'income', 'children']], diagonal='hist')
plt.show()
# 4) 按区域展示平均收入的柱状图, 并显示标准差
region_income_mean = data.groupby('region')['income'].mean()
region_income_std = data.groupby('region')['income'].std()
region_income_mean.plot(kind='bar', yerr=region_income_std, rot=0, title='不同区域平均收入柱状图')
plt.xlabel('Region')
plt.show()
# 5) 多子图绘制: 账户中性别占比饼图, 有车的性别占比饼图, 按孩子数的账户占比饼图
sex_data = data.groupby(['sex'])['sex'].count()
car_data = data[data['car'] == 'YES'].groupby(['sex'])['sex'].count()
children_data = data.groupby(['children'])['children'].count()
fig = plt.figure(figsize = (7,6))
fig.add_subplot(2,2,1)
```

```
sex_data.plot(kind = 'pie',title = 'Customer Sex',startangle = 60,autopct = '%1.1f%%')
fig.add_subplot(2,2,2)
car_data.plot(kind = 'pie',title = 'Customer Car Sex',startangle = 60,autopct = '%1.1f%%')
fig.add_subplot(2,2,3)
children_data.plot(kind = 'pie',title = 'Customer Children',startangle = 60,autopct = '%1.1f%%')
plt.savefig('饼图.jpg',dpi = 400,bbox_inches = 'tight')
plt.show()
```

6) 各性别收入的箱须图

```
data.boxplot(column='income', by='sex')
```

调整子图布局

```
plt.tight_layout()
```

显示图形

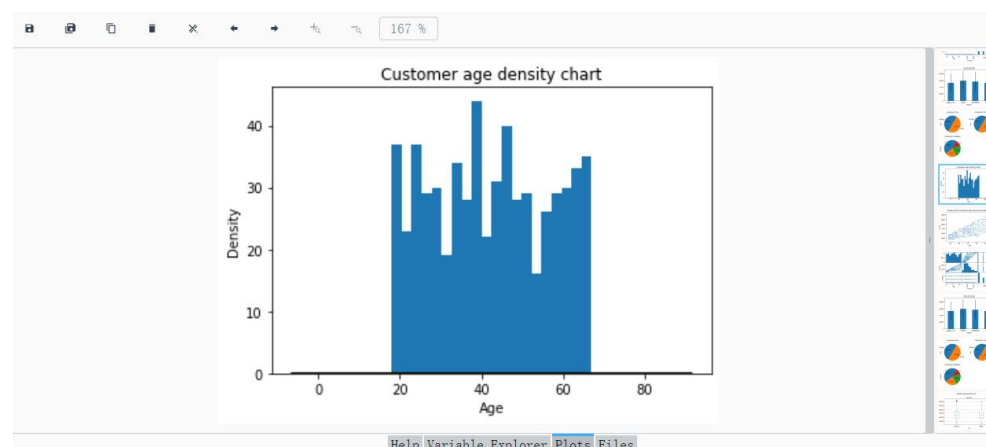
```
plt.show()
```

程序说明

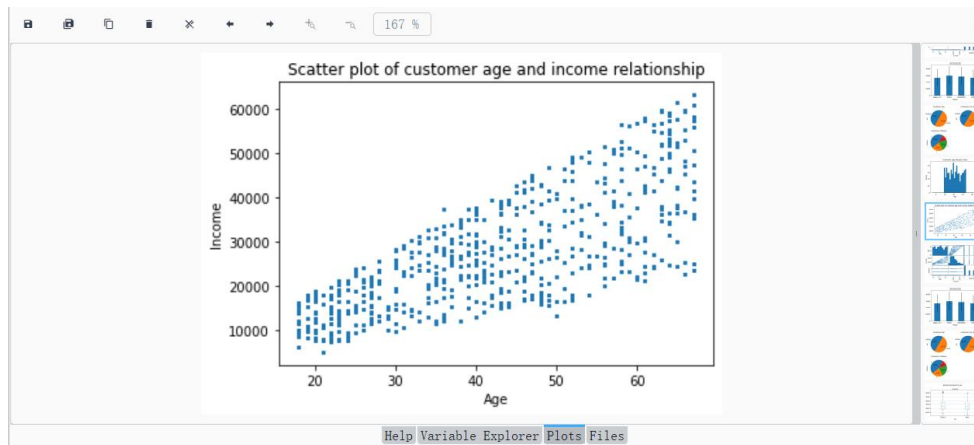
1. 读取数据：首先，程序读取名为 'bankpep.csv' 的数据文件，该文件包含了银行客户的信息。
2. 创建子图：接着，程序创建一个包含两行三列的图形，用于显示不同的图表。这些子图包括：
3. 不同区域平均收入柱状图：通过分组银行客户数据，计算不同区域的平均收入，并使用柱状图展示这些平均值。柱状图上的误差条表示了收入的标准差。
4. 性别占比饼图：
 - 账户中性别占比：展示了所有账户中不同性别的占比。
 - 有车的性别占比：展示了拥有车辆的账户中不同性别的占比。
 - 按孩子数的账户占比：展示了不同孩子数量的账户的占比。
5. 各性别收入的箱须图：绘制了各性别客户的收入箱须图，以展示收入的分布情况。
6. 调整子图布局：最后，程序通过调整子图布局以确保图形的可读性。
7. 显示图形：使用 'plt.show()' 函数显示生成的图形。

程序截图

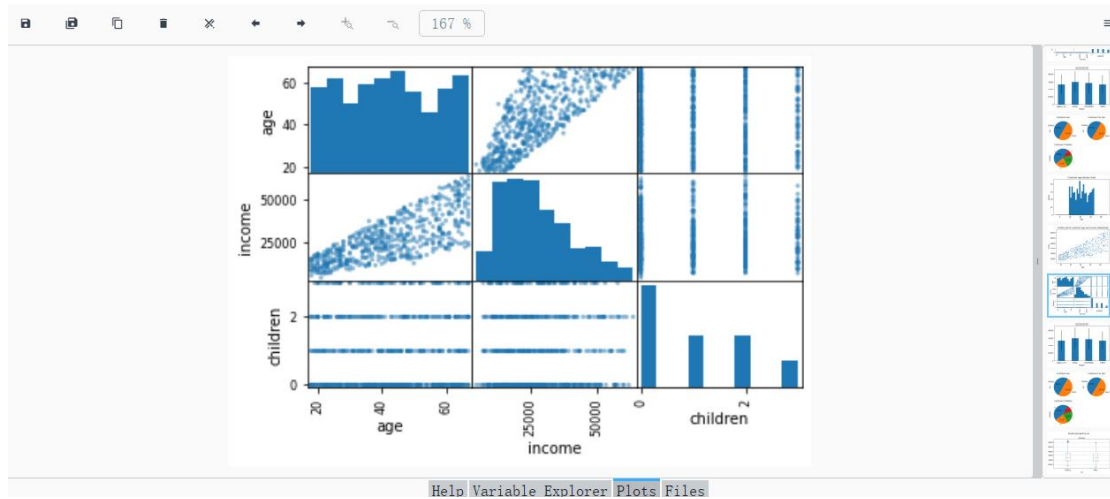
1) 客户年龄分布的直方图和密度图



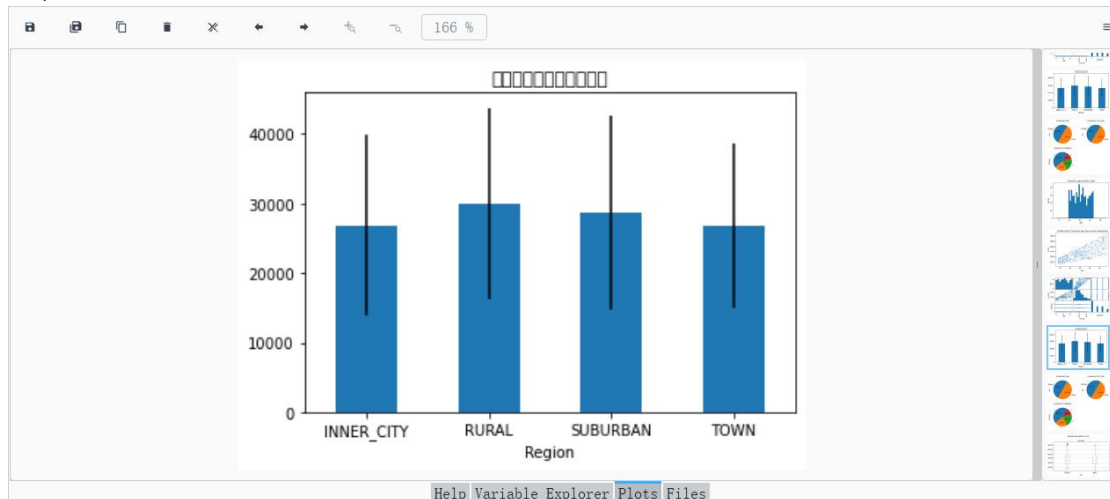
2) 客户年龄和收入关系的散点图



3) 散点图观察账户（年龄，收入，孩子数）之间的关系，对角线显示直方图



4) 按区域展示平均收入的柱状图，并显示标准差



5) 多子图绘制：账户中性别占比饼图，有车的性别占比饼图，按孩子数的账户占比饼图



6) 各性别收入的箱须图

